

Highlights

Convention-aware and reproducibly validated biodiversity computation for ecological informatics

Matthew Roughan

- Biodiversity index conventions are exposed as machine-readable meta-data.
- Diversity and similarity indices are integrated in one auditable workflow.
- Executable reference cases link formulas, tests, documentation, and benchmarks.
- Workflow audits combine coverage diagnostics and bootstrap uncertainty reports.
- Input validation is separated from computation via a **Validated** wrapper, enabling safe pipeline reuse and fair cross-language benchmarks.
- Warmed cross-language benchmarks report both validated and pre-validated pathways.

Convention-aware and reproducibly validated biodiversity computation for ecological informatics

Matthew Roughan^a

^a*Affiliation to be supplied, ,*

Abstract

Biodiversity indices are central to ecological informatics, microbial ecology, conservation assessment, and ecosystem monitoring, yet many commonly used indices are convention-sensitive. Names such as Simpson, Shannon, Jaccard, and Sorensen can refer to different transformations, logarithm bases, input types, or similarity-versus-dissimilarity conventions across software ecosystems. These differences create a reproducibility problem for data-intensive ecological workflows, where alpha-diversity summaries, estimator diagnostics, and pairwise dissimilarity matrices are often combined in downstream analyses. We present `DiversityAndDissimilarity.jl`, a Julia package designed as an ecoinformatics infrastructure layer for reproducible diversity computation. Its three core contributions are: (1) a convention-aware metadata and trait system that exposes index families, input modes, output modes, ranges, aliases, and metric properties as machine-readable data; (2) an executable validation and benchmark corpus that links formula-level reference cases, cross-package migration checks, documentation, figures, and performance reports; and (3) workflow audit reports that combine alpha-diversity summaries, coverage diagnostics, pairwise distances, and bootstrap uncertainty intervals. The package treats diversity and similarity indices as related computational objects because they share data orientation, normalization, incidence, abundance, and labeling assumptions. Benchmarks against R and Python demonstrate that reproducibility-oriented metadata can coexist with optimized matrix kernels. The result is not only a library of biodiversity indices, but a framework for making biodiversity calculations inspectable, testable, and portable across ecological data workflows.

Keywords: biodiversity informatics, ecological informatics, diversity indices, beta diversity, reproducible research, Julia

1. Introduction

Ecological informatics is concerned with the computational representation, analysis, and reuse of ecological data. Biodiversity measurement is one of its recurring building blocks: richness, Shannon entropy, Simpson-family measures, Hill numbers, and pairwise similarities or dissimilarities are used to summarize communities, compare sites, diagnose sampling effort, and support ordination, clustering, monitoring, and conservation decisions (Hill, 1973; Jost, 2006; Chao, 1984; Whittaker, 1960; Legendre and Legendre, 2012). As ecological datasets grow larger and more heterogeneous, these calculations increasingly occur inside automated workflows rather than as isolated hand calculations.

That automation exposes a persistent problem: biodiversity indices are not only mathematical formulas, but also software conventions. “Simpson diversity” can mean concentration, its complement, or its reciprocal. Shannon entropy can be reported in nats, bits, or as an effective number after exponentiation. Jaccard and Sorensen–Dice can refer to binary incidence similarities, abundance-sensitive variants, or their distance complements. These differences are often well documented within individual software ecosystems, but documentation written as prose is easy to miss when results are passed between R, Python, Julia, spreadsheets, notebooks, reports, and data repositories.

The consequence is an ecoinformatics reproducibility problem. A numerical value may be correct within one package and misleading when interpreted under another package’s naming convention. A distance matrix may be generated from relative abundances in one workflow and incidence data in another. An index may be treated as a metric by downstream algorithms even when it does not satisfy metric assumptions. These problems are subtle because they need not produce errors: they produce plausible but differently defined results.

We present `DiversityAndDissimilarity.jl`, a Julia package designed to make these conventions explicit and executable. The package provides scalar, matrix, and table-oriented biodiversity calculations, but its main novelty is not the existence of another index library. Instead, the package treats index definitions, cross-package conventions, validation examples, benchmark results, and workflow diagnostics as connected computational artifacts. This orientation aligns with the broader goals of reproducible and reusable research software (Peng, 2011; Wilkinson et al., 2016).

The paper makes three contributions. First, we introduce a convention-aware metadata and trait layer for diversity and similarity indices. Index objects report their family, input mode, output mode, range, aliases, metric status, and availability of optimized matrix kernels. Second, we introduce an executable validation and benchmark corpus. Formula-level reference cases, cross-package migration checks, documentation examples, figures, and benchmark tables are generated from the same package-controlled sources. Third, we introduce workflow audit reports that keep estimator diagnostics, coverage checks, pairwise matrices, and bootstrap uncertainty intervals tied to the same labels and input data. The paper is therefore framed around reproducible biodiversity computation rather than around a static software description.

2. Background and Motivation

2.1. *Convention-sensitive biodiversity computation*

The need for explicit conventions is clearest in the Simpson family. The concentration

$$\lambda = \sum_i p_i^2$$

is a dominance measure: larger values indicate greater dominance by common classes. The Gini–Simpson form, $1 - \lambda$, reverses that interpretation. The inverse Simpson form, $1/\lambda$, reports an effective diversity. All three are legitimate, but a plot, table, or downstream model that labels all of them as “Simpson” is underspecified.

Similar ambiguity occurs for entropy and pairwise indices. Shannon entropy depends on the logarithm base, and Hill-number transformations change the scale from entropy to effective diversity (Hill, 1973; Jost, 2006). Jaccard and Sorensen–Dice indices depend on whether input data are interpreted as incidence or abundance. Bray–Curtis, Hellinger, and related dissimilarities require choices about normalization and matrix orientation. These choices matter for ecological interpretation and for algorithmic compatibility.

2.2. *Why diversity and similarity indices belong together*

Software often separates alpha-diversity functions from similarity or dissimilarity functions. The distinction is ecologically meaningful: single-sample richness and entropy summarize within-sample diversity, whereas

Bray–Curtis, Jaccard, Sorensen–Dice, and Hellinger comparisons describe between-sample relationships. In data analysis, however, they are usually computed from the same sample-by-taxon matrix and reported together. A monitoring workflow may calculate richness and Shannon entropy per site, then construct a Bray–Curtis matrix for ordination or clustering. A microbiome workflow may report alpha-diversity summaries beside incidence-based beta-diversity comparisons.

Putting these calculations in one package has three advantages for ecological informatics. First, it makes shared assumptions visible. Incidence Jaccard and richness both depend on non-zero support; Bray–Curtis and Simpson-family indices are abundance-sensitive; Hellinger distances depend on normalized profiles. Second, it protects workflow integrity. Labels, sample ordering, matrix orientation, and distance-versus-similarity conventions are tracked in one computational setting rather than reconstructed manually. Third, it makes extension systematic. A new index can participate in scalar calculations, matrix kernels, reference validation, documentation, and benchmark reporting through the same dispatch and metadata interface.

3. Software Design

3.1. *Convention-aware index metadata*

`DiversityAndDissimilarity.jl` represents indices as small dispatchable objects and exposes their interpretation through trait-like metadata functions. For each supported index, the package records the index family, expected input mode, output mode, numerical range, interpreted bounds, metric status, aliases, convention notes, and whether a specialized matrix kernel is available. Descriptor functions expose finiteness, boundedness, non-negativity, symmetry, triangle-inequality status, metric and weaker metric-like classifications, and whether an index is conventionally a similarity or a dissimilarity. Unknown or unencoded classifications are reported explicitly rather than silently promoted to a Boolean claim. Figure 1 summarizes the design.

For example, `Simpson()` is defined as concentration, while `GiniSimpson()` is the complement used by common R workflows; this matches the `simpson` option in `vegan::diversity` (Oksanen et al., 2022). The metadata layer makes that distinction available to users and to programs. A report generator can label an output as a concentration, a similarity, or a dissimilarity. A workflow audit can warn when an index is not a metric. A documentation

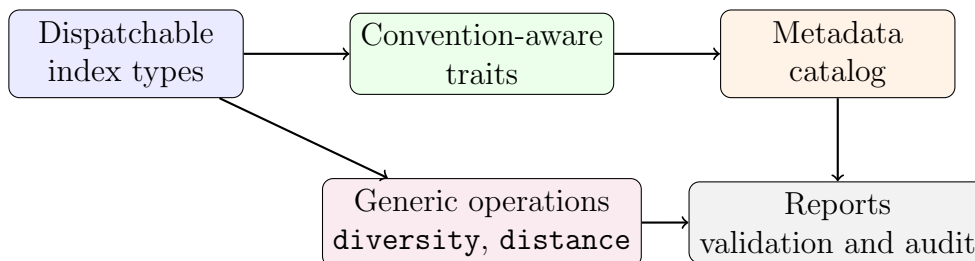


Figure 1: The package separates index identity, convention metadata, and numerical evaluation. This supports generic workflows while keeping naming and output conventions explicit.

page can group indices by input mode rather than by informal name. Bound metadata also records the interpretation of endpoints: for a similarity, the lower endpoint usually denotes no overlap, while for a dissimilarity it usually denotes identical or indistinguishable inputs.

The design is intentionally lightweight. Metadata functions return ordinary Julia values such as symbols, named tuples, and strings. This avoids a heavy ontology while still making convention information inspectable and machine-readable. Multiple dispatch selects calculations; metadata functions describe interpretation. The result is a separation between the numerical method and the semantic commitments that travel with it.

3.2. Executable validation corpus

The second software-design component is an executable validation corpus. The package exposes formula-level and cross-package reference cases through the public API. These cases include small examples for Shannon entropy, Simpson-family transformations, Bray–Curtis dissimilarity, incidence Jaccard, Sorensen–Dice, and Pielou evenness. Each reference case has an expected value, tolerance, and convention label. Figure 2 shows how these cases connect tests, documentation, benchmarks, and manuscript artifacts.

This corpus serves a different role from ordinary unit tests. Unit tests guard implementation details; reference cases communicate meaning. They are small enough to inspect by hand and structured enough to evaluate automatically. They also support migration from established tools. A user moving from **vegan**, for example, can verify that Shannon, Gini–Simpson, inverse Simpson, and Bray–Curtis examples match the intended R conventions before translating a larger workflow.

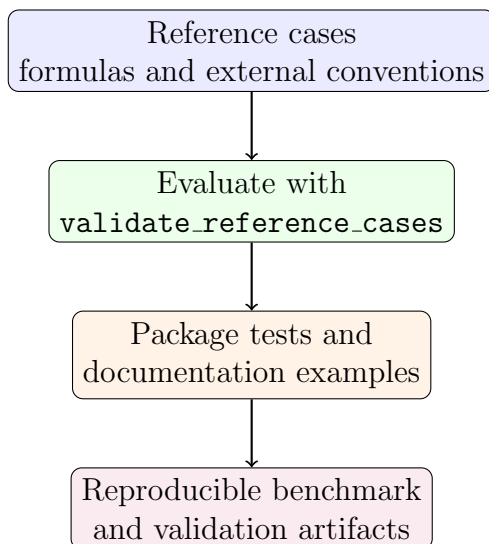


Figure 2: The validation corpus makes convention choices executable and visible in tests, documentation, and paper artifacts.

3.3. Workflow helpers and audit reports

The package includes workflow-level helpers because reproducibility failures often occur between calculations rather than inside a single formula. The **estimator_report** function reports observed richness, singleton and doubleton counts, sample coverage, Shannon estimator results, effective diversities, and warnings. This makes estimator assumptions visible when sample size and unseen taxa may influence interpretation (Chao and Shen, 2003). Low-sample corrections for entropy and information divergences are grounded in the same sampling problem. The plugin entropy estimator has a systematic finite-sample bias (Paninski, 2003); Miller–Madow-style corrections address the leading entropy-term bias, whereas pseudocount and shrinkage estimators trade bias for finite estimates when categories are sparse. For unseen categories, Good–Turing coverage estimates (Good, 1953) and later unseen-estimation work (Valiant and Valiant, 2017) motivate explicit unseen-mass corrections. Minimax results for large alphabets (Jiao et al., 2015; Wu and Yang, 2016) show why naive plugin estimators can be fragile when support size is comparable with sample size. Bayesian entropy estimators such as the NSB prior (Nemenman et al., 2002) provide another route to low-sample information-functional estimation, and motivate exposing Bayesian smoothing options for KL and Jensen–Shannon-style divergences.

The `diversity_audit` function combines alpha summaries, estimator diagnostics, labeled pairwise matrices, and warnings about potentially problematic inputs. It is designed as an early-stage diagnostic for ecological data analysis: users can check row totals, coverage, labels, and convention-sensitive distance choices before moving to ordination, clustering, or modelling.

The `uncertainty_audit` function extends this workflow layer by summarizing bootstrap uncertainty for Shannon entropy and effective Shannon diversity. It reports per-sample estimates, standard errors, percentile intervals, sample coverage, and labels. The goal is not to replace a full model-based analysis of uncertainty, but to make early uncertainty diagnostics available at the same stage as convention and coverage checks. This is useful in ecological informatics workflows because uncertainty is often lost when alpha summaries and distance matrices are exported as separate tables.

3.4. *Validated input pathway*

A recurring tension in defensive scientific software is the cost of input validation. Checks for non-negative abundances, finite values, and positive row totals protect users from silent errors, but they also add overhead that is paid on every function call. In pipeline workflows where the same community matrix is passed to richness, entropy, alpha-diversity summary, and distance matrix functions in sequence, this overhead accumulates and can account for a meaningful fraction of total compute time, particularly for inexpensive row-wise operations such as richness counting.

`DiversityAndDissimilarity.jl` addresses this tension through an explicit separation of validation from computation. Calling `validate(community)` runs all precondition checks once and returns a `Validated{T}` wrapper value. Subsequent calls on the wrapper skip per-call validation entirely. The default, safe pathway — passing raw data directly to any function — continues to validate on every invocation.

This design makes the validation boundary a first-class concept rather than an invisible implementation detail. The `Validated` dispatch methods contain only computation; the corresponding raw-data dispatch methods validate and then delegate to the `Validated` version. A user who writes `v = validate(community)` and then calls `richness(v)`, `shannon_entropy(v)`, and `bray_curtis_distance(v)` declares that validation has been performed once, and the type system enforces that commitment across all downstream calls.

Constructing `Validated(data)` directly without calling `validate` bypasses all checks and is explicitly documented as unsafe. This risk is localised: ordinary use of the public API remains fully protected by default.

The validated pathway also corrects a methodological asymmetry in cross-language benchmarking. Python/NumPy and R/`vegan` routines do not re-validate input arrays on each call, so comparing Julia’s safe pathway against those tools overstates Julia’s overhead for repeated computation on the same matrix. The benchmark corpus therefore reports both pathways: task names ending in `_prevalidated` use `Validated` data and are the fair computational comparison; plain task names use raw data and reflect the full default user experience including validation.

4. Reproducibility and Benchmark Methods

Reproducibility is addressed at three levels: definitions, computations, and performance claims. Definition reproducibility is provided by the meta-data layer: each index object carries convention information that can be inspected where the calculation is made. Computational reproducibility is provided by the validation corpus: reference cases can be run by users, test suites, documentation builds, and manuscript-generation scripts. Performance reproducibility is provided by benchmark scripts that generate tabular results, figures, and environment metadata.

The benchmark corpus compares `DiversityAndDissimilarity.jl` with R `vegan` and Python/NumPy/SciPy on simulated community matrices. The benchmarks include richness, alpha-diversity summaries, Bray–Curtis distance matrices, Hellinger distance matrices, and incidence Jaccard matrices. The scripts record hardware and runtime metadata, including processor, operating system, Julia version, Julia thread count, Python interpreter, and R stack where available. Julia functions are warmed before timing so that reported values describe steady-state execution rather than just-in-time compilation. Very fast operations are timed using repeated inner calls to avoid zero-valued or timer-resolution-limited measurements.

The benchmark corpus reports two Julia pathways. The safe pathway passes raw data to each function, which validates on every call — the default user experience. The pre-validated pathway calls `validate` once on the community matrix before the timing loop; subsequent function calls on the `Validated` wrapper skip per-call validation. Python/NumPy and R `vegan` routines do not re-validate input arrays between calls, so the pre-validated

Julia pathway is the methodologically correct comparison for assessing computational performance. Both pathways are reported to make this distinction transparent rather than implicit.

The benchmarks are intentionally workflow-level comparisons. They include the costs of normalization, matrix traversal, allocation, and pairwise scaling because these costs are visible to users in ecological data analysis. The results are not claims about universal language performance; they are reproducible checks on the package’s implementation choices for common ecoinformatics tasks.

The repository also includes a cross-package validation data directory. It contains a simulated sample-by-taxon community matrix, hand-checkable paired vectors, and a manifest mapping validation tasks to equivalent `DiversityAndDissimilarity.jl`, R `vegan`, SciPy, and scikit-bio-style calls. These files are intended as a seed for a larger biodiversity software concordance suite. They separate the validation data from the package-specific function calls and from the convention being tested.

5. Results

5.1. Reproducible convention checks

The executable reference corpus currently validates scalar alpha-diversity indices and pairwise similarity/dissimilarity indices against formula-level and cross-package expectations. Its main practical result is that conventions become testable objects. A documentation statement such as “`GiniSimpson()` follows the R `vegan` Simpson convention” is backed by an executable case rather than only by prose. Optimized kernels and table-oriented helpers can therefore be checked against the same small reference cases used to explain the package to users.

5.2. Benchmark results

Selected warmed per-call benchmark results are shown in Table 1. The table reports two Julia pathways alongside Python and R/`vegan`: *Julia (safe)* validates on every call, which is the default user experience; *Julia (pre)* calls `validate` once before the timing loop, matching the Python and R calling convention. The pre-validated column is therefore the methodologically fair cross-language comparison for computational performance.

The validation overhead is most visible for inexpensive row-wise operations. For richness at the default scenario size (400 sites, 200 taxa), the

Table 1: Selected warmed, per-call benchmark results on Intel(R) Core(TM) Ultra 9 285K with Julia 1.12.6 using 1 Julia thread(s). Times in seconds. *Julia (safe)* validates on every call (the default user experience); *Julia (pre)* calls `validate` once before the timing loop, matching the Python and R calling convention. Dashes indicate no direct equivalent in that stack.

Scenario	Task	Julia (safe)	Julia (pre)	Python	R/ <i>vegan</i>
default	richness	5.8×10^{-5}	5.87×10^{-6}	2.88×10^{-5}	4×10^{-4}
default	Shannon entropy	2.99×10^{-4}	2.45×10^{-4}	7.91×10^{-4}	1.75×10^{-3}
default	Bray–Curtis matrix	5×10^{-3}	4.94×10^{-3}	6.06×10^{-3}	8.15×10^{-3}
default	Jaccard matrix	3.33×10^{-4}	2.56×10^{-4}	1.45×10^{-2}	9.4×10^{-3}
default	Hellinger matrix	5.25×10^{-3}	5.16×10^{-3}	8.31×10^{-2}	—
large	Bray–Curtis matrix	7.31×10^{-2}	7.15×10^{-2}	8.37×10^{-2}	1.29×10^{-1}
large	Jaccard matrix	3.39×10^{-3}	2.87×10^{-3}	1.98×10^{-1}	1.38×10^{-1}

safe pathway is slower than NumPy because the input is fully checked on every call, while the pre-validated pathway is substantially faster. For heavier matrix operations such as Bray–Curtis and Hellinger distance matrices, validation cost is negligible relative to the $O(n_{\text{sites}}^2 \cdot n_{\text{taxa}})$ computation, and both pathways outperform or match the reference stacks. Incidence Jaccard distance matrices benefit from packed `UInt64` bitsets that reduce pairwise comparisons to word-wise intersections and population counts, yielding a larger margin over SciPy and R/*vegan* at scale.

These results demonstrate that the additional metadata and validation machinery does not prevent efficient matrix implementations. Reproducible performance reporting is important precisely because different tasks are dominated by different cost structures: some by input validation, some by memory access patterns, and others by pairwise matrix construction. Benchmarking guided targeted implementation improvements, including column-major matrix traversal and bitset incidence kernels, while the reference corpus checked that optimized methods preserved the intended conventions.

5.3. Workflow case study with uncertainty audit

We added a simulated forest-gradient workflow case study based on the cross-package validation matrix. The case study reads a sample-by-taxon matrix with habitat and gradient metadata, runs `diversity_audit` for alpha summaries and a labeled Bray–Curtis distance matrix, and then runs `uncertainty_audit` to bootstrap Shannon entropy and effective diversity for each plot. Table 2 shows the generated audit table.

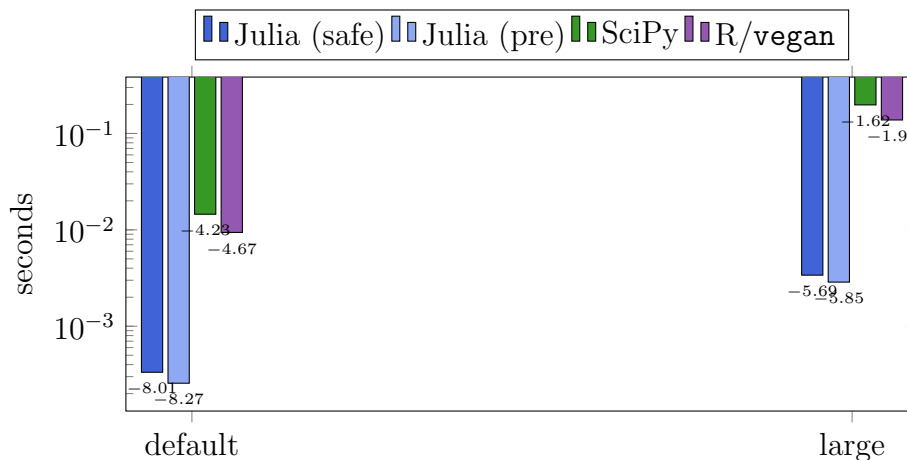


Figure 3: Jaccard distance matrix timings after bitset incidence encoding. *Julia (safe)* includes per-call validation; *Julia (pre)* validates once before the loop, matching the SciPy and R/*vegan* calling convention. The benchmark is workflow-level and hardware-dependent, but illustrates the algorithmic benefit of packed incidence representation.

This example illustrates the package’s third contribution: uncertainty, coverage, labels, and pairwise comparisons remain part of one reproducible workflow. The case study report is generated by a repository script, and the LaTeX table used here is produced from the same script. As with the benchmark section, the manuscript artifact is downstream of executable package artifacts rather than manually copied results.

6. Discussion

6.1. Contribution to ecological informatics

`DiversityAndDissimilarity.jl` contributes to ecological informatics by treating biodiversity calculation as an information-management problem as well as a numerical one. The package does not merely compute indices; it records and exposes the interpretive context needed to reuse those computations. This is important for data-intensive ecology, where biodiversity summaries are increasingly embedded in automated pipelines, dashboards, teaching materials, benchmark studies, and reproducible research compendia.

The integration of diversity and similarity indices is central to this contribution. Alpha-diversity and beta-diversity calculations differ ecologically,

Table 2: Selected workflow case-study audit results. Shannon values are bootstrap estimates with percentile intervals from `uncertainty_audit`.

Site	Habitat	Richness	Coverage	Shannon H	Effective D
plot_A	forest	7	0.979	2.31 [1.85, 2.509]	4.96 [3.704, 5.6]
plot_B	forest	8	0.912	2.443 [1.795, 2.657]	5.438 [3.659, 6.147]
plot_C	forest	8	0.923	2.686 [2.054, 2.81]	6.437 [4.183, 7.021]
plot_D	forest	7	0.923	2.439 [1.834, 2.599]	5.423 [3.557, 6.201]
plot_E	wetland	8	0.939	2.575 [2.017, 2.747]	5.959 [3.952, 6.537]
plot_F	wetland	8	0.951	2.504 [2.007, 2.712]	5.671 [3.925, 6.469]
plot_G	grassland	8	0.909	2.457 [1.902, 2.634]	5.491 [3.778, 6.454]
plot_H	grassland	7	0.941	2.352 [1.847, 2.557]	5.107 [3.622, 5.714]

but they share computational assumptions about matrix orientation, incidence, abundance, normalization, labels, and output mode. Placing them behind a shared metadata and validation interface helps users maintain coherence across the steps of a biodiversity workflow.

The workflow audit contribution extends the same idea to uncertainty. In many ecological analyses, point estimates are computed early and uncertainty is considered later, often after labels and conventions have already been transformed into intermediate files. By making bootstrap summaries, coverage diagnostics, and pairwise matrices available together, the package encourages users to identify low-coverage or high-uncertainty samples before committing to downstream analyses.

6.2. Limitations

The current package is focused on taxonomic diversity and common similarity/dissimilarity indices. It does not yet provide a comprehensive framework for phylogenetic diversity, functional diversity, rarefaction curves, uncertainty propagation across full workflows, sparse matrix kernels, or threaded large-scale distance calculations. The validation corpus is also deliberately small: it emphasizes inspectable convention examples rather than exhaustive coverage of all possible input types.

These limitations are useful because they identify the next layer of ecoinformatics value. The same metadata design could be extended to record phylogenetic trees, trait spaces, abundance transformations, compositional data assumptions, and estimator uncertainty. The same validation corpus could be expanded into a community-maintained benchmark and convention suite for biodiversity software.

6.3. Future work

Future work should prioritize features that increase scientific value rather than simply increasing the number of indices. High-value extensions include sparse and threaded matrix kernels for large monitoring datasets, uncertainty and bootstrap reports for estimator workflows, first-class support for phylogenetic and functional diversity, and a richer cross-package validation suite covering R, Python, and Julia biodiversity tools. These additions would make the package a stronger platform for computational biodiversity research and a more compelling contribution to ecological informatics.

7. Conclusion

Reproducible biodiversity analysis requires more than correct formulas. It requires explicit conventions, executable validation, portable workflow diagnostics, and performance reports that can be regenerated as software and hardware change. `DiversityAndDissimilarity.jl` provides these elements through a convention-aware metadata layer and an executable validation and benchmark corpus. By treating diversity and similarity indices as related computational objects, the package supports coherent, auditable biodiversity workflows for ecological informatics.

Data and code availability

The software, validation cases, benchmark scripts, documentation, manuscript source, validation datasets, workflow case-study scripts, and figure-generation artifacts are available in the `DiversityAndDissimilarity.jl` source repository. Benchmark tables, case-study tables, and figures used in this manuscript are generated from repository scripts rather than hand-entered values.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used Claude Code (claude-sonnet-4-6, Anthropic) to assist with code generation, refactoring, documentation, benchmarking, and manuscript drafting. After using this tool, the author reviewed and edited the content as necessary and takes full responsibility for the content of the published article.

Declaration of competing interest

The author declares no competing interests.

Funding

No specific funding statement has been supplied for this draft.

CRedit authorship contribution statement

Matthew Roughan: Conceptualization, Software, Methodology, Validation, Writing – original draft, Writing – review and editing.

References

- Chao, A., 1984. Nonparametric estimation of the number of classes in a population. *Scandinavian Journal of Statistics* 11, 265–270.
- Chao, A., Shen, T.J., 2003. Nonparametric estimation of shannon’s index of diversity when there are unseen species in sample. *Environmental and Ecological Statistics* 10, 429–443. doi:10.1023/A:1026096204727.
- Good, I.J., 1953. The population frequencies of species and the estimation of population parameters. *Biometrika* 40, 237–264. doi:10.1093/biomet/40.3-4.237.
- Hill, M.O., 1973. Diversity and evenness: A unifying notation and its consequences. *Ecology* 54, 427–432. doi:10.2307/1934352.
- Jiao, J., Venkat, K., Han, Y., Weissman, T., 2015. Minimax estimation of functionals of discrete distributions. *IEEE Transactions on Information Theory* 61, 2835–2885. doi:10.1109/TIT.2015.2412945.
- Jost, L., 2006. Entropy and diversity. *Oikos* 113, 363–375. doi:10.1111/j.2006.0030-1299.14714.x.
- Legendre, P., Legendre, L., 2012. *Numerical Ecology*. 3 ed., Elsevier, Amsterdam.
- Nemenman, I., Shafee, F., Bialek, W., 2002. Entropy and inference, revisited, in: *Advances in Neural Information Processing Systems*, pp. 471–478.

- Oksanen, J., Simpson, G.L., Blanchet, F.G., Kindt, R., Legendre, P., Minchin, P.R., O'Hara, R.B., Solymos, P., Stevens, M.H.H., Szoecs, E., Wagner, H., Barbour, M., Bedward, M., Bolker, B., Borcard, D., Carvalho, G., Chirico, M., De Caceres, M., Durand, S., Evangelista, H.B.A., FitzJohn, R., Friendly, M., Furneaux, B., Hannigan, G., Hill, M.O., Lahti, L., McGlinn, D., Ouellette, M.H., Ribeiro Cunha, E., Smith, T., Stier, A., Ter Braak, C.J.F., Weedon, J., 2022. *vegan*: Community ecology package. R package.
- Paninski, L., 2003. Estimation of entropy and mutual information. *Neural Computation* 15, 1191–1253. doi:10.1162/089976603321780272.
- Peng, R.D., 2011. Reproducible research in computational science. *Science* 334, 1226–1227. doi:10.1126/science.1213847.
- Valiant, G., Valiant, P., 2017. Estimating the unseen: Improved estimators for entropy and other properties. *Journal of the ACM* 64, 37:1–37:41. doi:10.1145/3125643.
- Whittaker, R.H., 1960. Vegetation of the siskiyou mountains, oregon and california. *Ecological Monographs* 30, 279–338. doi:10.2307/1943563.
- Wilkinson, M.D., Dumontier, M., Aalbersberg, I.J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.W., da Silva Santos, L.B., Bourne, P.E., Bouwman, J., Brookes, A.J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C.T., Finkers, R., Gonzalez-Beltran, A., Gray, A.J.G., Groth, P., Goble, C., Grethe, J.S., Heringa, J., 't Hoen, P.A.C., Hooft, R., Kuhn, T., Kok, R., Kok, J., Lusher, S.J., Martone, M.E., Mons, A., Packer, A.L., Persson, B., Rocca-Serra, P., Roos, M., van Schaik, R., Sansone, S.A., Schultes, E., Sengstag, T., Slater, T., Strawn, G., Swertz, M.A., Thompson, M., van der Lei, J., van Mulligen, E., Velterop, J., Waagmeester, A., Wittenburg, P., Wolstencroft, K., Zhao, J., Mons, B., 2016. The fair guiding principles for scientific data management and stewardship. *Scientific Data* 3, 160018. doi:10.1038/sdata.2016.18.
- Wu, Y., Yang, P., 2016. Minimax rates of entropy estimation on large alphabets via best polynomial approximation. *IEEE Transactions on Information Theory* 62, 3702–3720. doi:10.1109/TIT.2016.2548468.